

How Much Adaptation Do World Models Need?

A Study of Domain Shifts in Doom

Rohan Nagabhirava
Carnegie Mellon University
Pittsburgh, PA, United States
rnagabhi@andrew.cmu.edu

Keerthana Chirumamilla
Carnegie Mellon University
Pittsburgh, PA, United States
kchiruma@andrew.cmu.edu

Changliu Liu
Carnegie Mellon University
Pittsburgh, PA, United States
cliu6@andrew.cmu.edu

Abstract: World models have become capable simulators, but they are evaluated almost exclusively on held-out rollouts from the domain they were trained on. A small domain shift, such as a new map of the same game, a new texture set, a lighting change or a different cast of enemies, makes their outputs drift and lose realism, though the underlying physics and game engine are exactly the same. With DoomShift, a controlled version of this shift in Doom, we measure what it breaks and how much adaptation it takes to repair. We train three diffusion backbones, initialized from pretrained diffusion image models, on four maps of Doom and score each on in-domain data and on 13 other out-of-distribution maps that it never saw. All three keep their response to the controls and lose the scene: the U-Net’s perceptual error (LPIPS) doubles, from 0.158 to a median 0.303, its scene PSNR drops 2.9 dB, from 25.2 to a median 22.3 dB, and two transformers of 0.6 and 2.3 B parameters lose as much. Very little adaptation repairs it: rank-16 adapters trained on 8 episodes of each unseen map with 2% of the original training updates recover 72% of the lost perceptual quality and about half of the lost PSNR on the U-Net, and 76% and 77% of the perceptual loss on the two transformers.

Keywords: World models, Few-shot adaptation

1 Introduction

Game world models now render playable worlds from pixels and controls. GameNGen runs Doom inside a diffusion model [1], DIAMOND learns Atari and Counter-Strike in pixel space [2] and Matrix-Game 2 scales the recipe to open worlds [3]. Each of these models is evaluated only on held-out rollouts from the environment it was trained on: the reported score is a held-out trajectory drawn from its own training scenes and none has been shown to generalize to a new environment within the same game. This holds even for MultiGen, which trains a single Doom model across 100 procedurally generated maps with a pre-trained agent [4]: it reports PSNR, SSIM and LPIPS only on its training distribution, with no map held out for evaluation.

A learned simulator is useful only where it is faithful, and a robot or an agent moving to a new room, level or workspace needs to know how much of its world model still holds there and what it costs to make the rest hold. Game worlds let us pose this question with the engine held fixed and only the environment changed: a new map of the same game preserves the physics and the controls exactly, and only the layout, textures and lighting differ. Yet unchanged dynamics alone do not guarantee a faithful render: pretrained video and world models are typically moved to a new domain only through a further full fine-tune [5] or a dedicated adapter on the frozen backbone [6], and a model trained on a handful of maps tends to fall back on the appearance it already knows once the surroundings change. Pretraining one model on every map a deployment might meet is costly and never complete, and fine-tuning a separate model per map is prohibitively costly. We therefore explore what it takes to close the domain gap that opens when a world model meets a shift in which the underlying physics and game engine are exactly the same and only the surroundings, scenario or

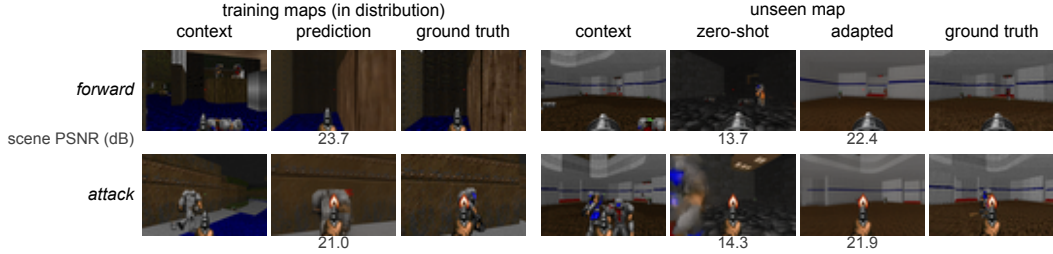


Figure 1: On an unseen map the U-Net still moves and fires but renders the training maps’ masonry; eight adaptation episodes restore the map. Left: training maps (in distribution). Right: unseen map 7, zero-shot and after adaptation.

scene change: what the model retains, what it loses, and how few episodes and updates recover the rest.

In short, a world model moved to a new map keeps playing the game but draws the wrong world, and a small adapter trained on eight episodes draws the map back (Figure 1). We make three contributions:

- **DoomShift, a benchmark that isolates a scene shift.** Seventeen Doom maps run on exactly the same physics and game engine and differ only in their surroundings. We build the pipeline end to end (the supplementary material): every-tic recordings, three diffusion world models initialized from pretrained diffusion image models and trained on four maps under one recipe, a decoder fine-tuned for the game, and a standard adapter for a new map, scored on the 13 maps the models never saw, zero-shot and after adaptation, against recorded frames and against each decoder’s reconstruction upper bound, which separates the model’s error from the renderer’s. Recordings, models, decoders and adapters are released (link withheld).
- **A scene shift breaks appearance, not control.** On the unseen maps all three models keep their turn response (directional score 0.80 to 0.81, against 0.84 to 0.86 on the training maps) while their perceptual error roughly doubles (LPIPS 0.158 to a median 0.303 for the U-Net) and their scene PSNR drops 2.8 to 3.3 dB. The loss is the same for a U-Net and two transformers from 0.6 to 2.3 B parameters, so it follows what the models were exposed to, not their architecture or scale. It is a step, not a slope: every unseen map is worse in LPIPS than every training map. And it lives in the model, since the decoder renders the unseen maps nearly as well as the training maps (Table 1, Figure 1).
- **How much adaptation it takes: very little.** A standard LoRA adapter on the frozen model (0.49 percent of the U-Net’s parameters), trained on eight episodes of the new map, closes half of the extra distance the shift opened to the upper bound in a median 150 updates. By 4k updates, 2 percent of pretraining and about one A4000 GPU-hour, it closes a median 94 percent of that distance and recovers 72 percent of the lost perceptual quality. The same procedure recovers 76 and 77 percent on PixArt- α and SD 3.5. A full fine-tune with 205 times the trainable parameters recovers 87 percent against the adapter’s 71 on four maps (Tables 1 and 2, Figure 2).

2 Related work

Held-out evaluation of world models exists, but it holds out something other than a map of the same game: a new robot body [7], a new game or environment [6, 5] or a new building [8]. Doom maps have been held out, but only to test the agents that play them [9, 10], and game world models that do show new scenes generate them from an image, with no recorded frames to score against [11, 12]. To our knowledge, no published game world model has been scored against recorded frames on maps of its own game that were deliberately held out from training. Our adapter is not new: a small

adapter on a frozen video model moves it to a new setting in AVID [6] and adds action control in Vista [13], and AdaWorld adapts with a short full fine-tune [5], which we run as the comparator. None of them reports what adapting to one new scene costs in episodes and updates, which is what we measure.

3 Study design

Data. We collect the data for this research by running experiments with the Arnold agent [9] through ViZDoom [14] deathmatches: free-for-all matches where each player scores by killing the others. Arnold ships with 17 Doom maps; each has its own layout, textures and lighting. For every episode, we let Arnold play against 8 bots for 150 seconds. We record every tic, the engine’s 1/35 s simulation step, capturing a 320×240 frame and the 19 buttons the engine executed. We split the 17 maps into two groups. Four maps (2 to 5) are our training maps: we train on 500 episodes of each and score on 25 further validation episodes per map (512 windows). The other thirteen maps (1, 6 to 17) are never trained on: we record 24 new episodes on each, using 16 for adaptation and holding out 8 (256 windows) for scoring. In total we collect 2,412 episodes, about 96 hours of play and about 12.1 million frames.

Three backbones, one training recipe. We pick three pretrained backbones to ask three different questions. Our anchor is the SD 1.4 U-Net [15] (860M parameters): it is GameNGen’s backbone [1], so our in-distribution numbers come from the same network as the published Doom result. To test how different architectures perform at fixed scale, we swap in PixArt- α [16], a 628M transformer in the same latent space instead of a U-Net. To test how performance changes with scale, we use SD 3.5 Medium [17], a widely adopted open MMDiT (2.27B parameters, 16-channel latents). Its 16-channel latents give its decoder a higher reconstruction upper bound (31.8 against 28.6 dB on the training maps), so it can be the best of the three in distribution and still sit furthest below that bound.

We turn all three into next-tic world models under one shared recipe (the supplementary material). We give each model 32 context tics so it can see motion, train with v -prediction [18] for stable training across noise levels and add context noise augmentation as in GameNGen [1] so the model tolerates its own rollout errors. We train each backbone for 200k updates at batch size 32, keep an EMA at 0.9999 in fp32 and sample with 10-step DDIM [19], which we found to be a strong balance between quality and efficiency (sampler sweep in the supplementary material).

Metrics. We score every predicted frame against four questions: how accurate is it against the raw ground-truth frame, how realistic does it look, how far is it from the best the decoder could possibly render and does it turn the right way when we mirror the controls? We score the part of each frame that shows the world (rows 0 to 207) and leave out the HUD rows below it, which do not change with the map.

For accuracy, we compute PSNR against the raw frame, and for realism, LPIPS [20]. Absolute PSNR alone does not compare maps, because it rises with how static a recording is: copying the last context frame forward, the persistence baseline of video prediction [21], already scores 19.8 dB on the unseen maps (18.5 to 22.5 per map). For distance from the best render, we read every map against two references: the *reconstruction upper bound* [22, 23], the decoder D applied to the ground-truth latent, which is in practice the best a prediction rendered through D can score (median 27.3 dB on the unseen maps, 28.6 on the training maps), and the in-distribution level, the same model on the training maps; the *gap to the upper bound* is the upper bound’s PSNR minus the prediction’s. For turning, we compute the *directional* score: the fraction of turning windows whose predicted view turns the other way once we swap left and right in the newest control.

Decoder. Every backbone is trained on latents from its frozen encoder, so the latent space is the contract between encoder, world model and decoder: changing the encoder would move every training target and invalidate the trained backbones, the adapters and the reconstruction upper bound. The decoder only renders: it sits after the model, and its own error is what the reconstruction upper bound measures, so fine-tuning it changes what we can see, not what the model knows. We fine-tune the

Table 1: Scene PSNR and LPIPS one tic ahead against the raw frame, the reconstruction upper bound of each backbone’s fine-tuned decoder, and the directional score: training maps (pooled over 512 validation windows) and unseen maps (medians over the 13 unseen maps); directional pooled (ground-truth frames 0.885, 0.892).

Model	Maps	PSNR (dB)	LPIPS	Upper bound (dB)	Directional
SD 1.4 U-Net	training	25.20	0.158	28.55	0.855
	unseen	22.30	0.303	27.32	0.804
PixArt- α	training	25.18	0.159	28.55	0.842
	unseen	22.38	0.285	27.32	0.808
SD 3.5 Medium	training	25.38	0.126	31.84	0.840
	unseen	22.09	0.262	30.61	0.800

SD 1 decoder D on training-map frames paired with their frozen-encoder latents, using MSE plus 0.1 LPIPS for 3,486 updates and gating it on held-out training-map frames (the supplementary material); it never sees an unseen map. On 200 validation frames it raises the scene reconstruction from 26.9 to 28.4 dB and lowers its LPIPS from 0.092 to 0.060, and it reconstructs the unseen maps nearly as well as the training maps (median 27.3 against 28.6 dB), so the shift lives in the model, not the renderer. We use it for every pixel score of the U-Net and PixArt- α unless stated otherwise; SD 3.5 uses its own decoder, fine-tuned the same way.

Adaptation. For each unseen map we adapt each backbone’s 200k EMA weights with rank-16 LoRA [24] on every attention projection of the frozen backbone [13] and train the control MLP, input projection and noise-bucket embedding in full (4.2M parameters, 0.49 percent of the U-Net; 25.9M, 4.1 percent, of PixArt- α ; 40.6M, 1.8 percent, of SD 3.5) [25]. We train on 8 adaptation episodes (for the U-Net about 1 A4000 GPU-hour per map for 4k updates) and score the non-EMA weights at 0, 50, 100, 150, 250, 500, 1k, 2k, 4k and 8k updates (PixArt- α to 4k; SD 3.5 at 0, 250, 500, 1k, 2k and 4k), with the control checks (forgetting, the scene PSNR on the training maps, and the directional check) at 0 and 8k. 4k updates is the elbow of the curve, and we report it everywhere: across the 13 unseen maps it captures most of the benefit (on average 96 percent of each map’s 8k gain; Figure 2) at the smallest budget, which is why adaptation costs 2 percent of the original training updates; 8k serves as the check. As a comparator we fine-tune all 860.5M parameters of the U-Net at 2×10^{-5} on maps 6, 7, 8 and 16 with the same eight episodes and score it at 4k. A map’s budget is the first grid point at which it closes half of its excess gap over the in-distribution gap to the reconstruction upper bound, that is, at which its gap to the upper bound falls to the midpoint between its zero-shot gap and the training maps’ 3.4 dB; maps that do not by 8k are right-censored [26].

4 Results

The shift breaks the scene, not the controls. On unseen maps the turn response survives (Table 1). The appearance does not: the U-Net’s scene PSNR falls from 25.2 dB on the training maps to a median 22.3 dB, its LPIPS rises from 0.158 to 0.303, and its gap to the reconstruction upper bound grows from 3.4 to a median 5.0 dB. The three backbones lose the same things (Table 1; the supplementary material): scene PSNR drops 2.9, 2.8 and 3.3 dB and LPIPS rises by 0.145, 0.126 and 0.136, so the shift looks like a property of the task, not of the architecture or the scale. It is a step, not a slope: every unseen map has a worse LPIPS than every training map for all three backbones (the supplementary material). What the model keeps is how actions act on the world: zero-shot on map 7 the weapon fires and the view advances as in the ground truth (Figure 1) and the turn response holds; the loss sits in the scene, whose rows fall 2.9 dB (25.2 to a median 22.3) while the map-independent HUD rows fall 0.2 dB (29.7 to 29.5).

How much adaptation: updates, episodes, parameters. Eight episodes and 4k updates raise the median scene PSNR from 22.3 to 23.6 dB and lower the median LPIPS from 0.303 to 0.210 (Figure 2, Table 2); the 8k check adds little (23.7 dB, 0.204). Most of the PSNR still missing against the training maps (1.6 dB) reflects the unseen maps’ lower upper bound (27.3 against 28.6 dB), not

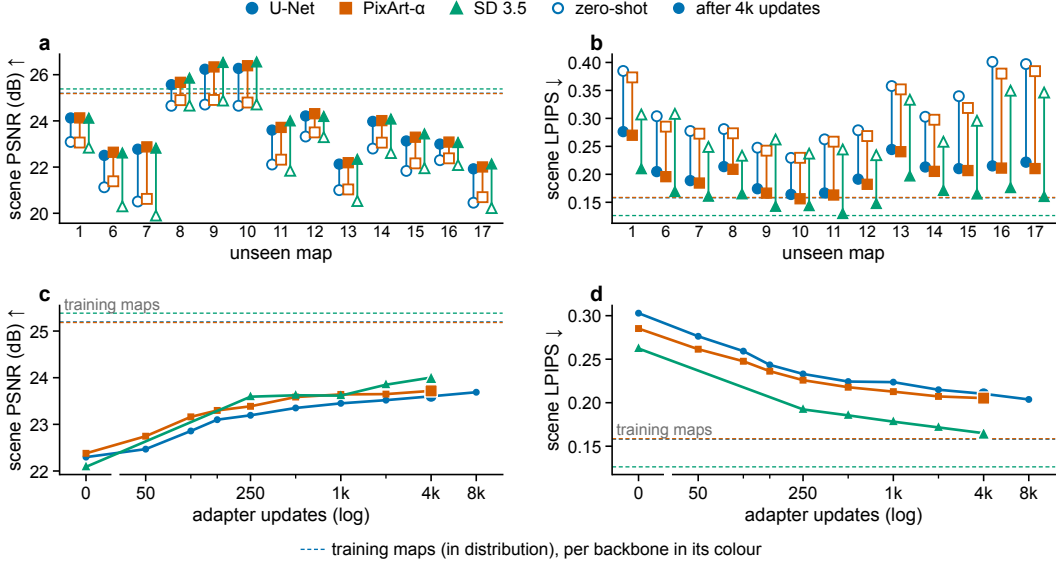


Figure 2: The shift and its repair, per unseen map and per backbone. (a) Scene PSNR and (b) LPIPS zero-shot (open) to 4k adapter updates (filled) for the U-Net, PixArt- α and SD 3.5, with each backbone’s in-distribution level on the training maps (dashed); (c, d) medians over the 13 unseen maps against updates.

the model. In domain the model sits 3.4 dB below its reconstruction upper bound; zero-shot on the unseen maps it sits a median 5.0 dB below, and after 4k adapter updates, 2 percent of the backbone’s 200k, a median 3.4 dB again, so per map the adapter closes a median 94 percent of the extra distance the shift opened, and at least half on every map. A map’s budget, the first of the grid reads (50, 100, 150, 250, 500, 1k, 2k, 4k and 8k updates) at which it closes half of that extra distance, has a median of 150 updates (50 to 2k per map, none censored), and half of each map’s 8k gain arrives within 100 updates on average. By the same 4k updates it recovers a median 72 percent of each map’s rise in LPIPS over the training maps (at least 48 percent) and a median 48 percent of its lost scene PSNR. The procedure carries over unchanged to the other backbones. On PixArt- α the adapter raises the median scene PSNR on the 13 unseen maps from 22.4 to 23.7 dB and lowers LPIPS from 0.285 to 0.205 by 4k: it recovers a median 76 percent of the rise in LPIPS and 48 percent of the lost PSNR and closes a median 101 percent of the extra distance to its upper bound (at least half on every map) with a median budget of 150 updates, against 72, 48 and 94 percent and 150 updates for the U-Net. On SD 3.5 the same adapter raises the median scene PSNR on the 13 unseen maps from 22.1 to 24.0 dB and lowers LPIPS from 0.262 to 0.165 by 4k: it recovers a median 77 percent of the rise in LPIPS over its in-distribution 0.126 (at least 54 percent) and 50 percent of the lost PSNR and closes a median 94 percent of the extra distance to its own upper bound (at least half on every map). Its median budget is 250 updates, but its grid starts at 250; read on the same grid, the U-Net’s median budget is also 250. A full fine-tune of all 860M parameters on the same eight episodes does better on each of the four comparator maps 6, 7, 8 and 16: at 4k it reaches a median scene PSNR of 23.1 dB and LPIPS of 0.182 against the adapter’s 22.9 dB and 0.209 (0.20 to 0.29 dB and 0.017 to 0.030 in LPIPS per map), leaves a median 3.3 dB to the upper bound against the adapter’s 3.5 and the training maps’ 3.4, and recovers a median 87 percent of the rise in LPIPS against the adapter’s 71. It trains 205 times as many parameters and needs a 49 GB RTX A6000, on which 4k updates take a median 0.8 hours, where the adapter trains on a 16 GB A4000 in 1.1 hours.

Table 2: What closing the gap takes. The U-Net’s LoRA by group of unseen maps, ordered by their zero-shot gap to the reconstruction upper bound, and over all 13; then the U-Net’s LoRA and full fine-tune on the four comparator maps, and the same LoRA procedure on PixArt- α and SD 3.5 over all 13 unseen maps. Medians over each row’s maps; non-EMA adapter weights; scene rows against the raw frame, each backbone through its fine-tuned decoder (SD 3.5 through its own, so its upper bound is higher); –: not scored (the PixArt- α and SD 3.5 grids stop at 4k). Budget: the first grid point at which a map closes half of its excess gap over its backbone’s in-distribution gap ($>8k$: censored; where a row’s two middle budgets differ, both are given).

Arenas (by zero-shot gap)	Scene PSNR (dB) $\uparrow$				Scene LPIPS $\downarrow$			Budget to half the excess gap
	zero-shot	4k	8k	upper bound	zero-shot	4k	8k	
Hard (7, 11, 14, 15)	21.97	23.37	23.43	27.74	0.290	0.199	0.193	1k–2k
Medium (8, 9, 10, 13, 17)	24.65	25.57	25.68	29.23	0.281	0.214	0.209	150
Easy (1, 6, 12, 16)	22.69	23.56	23.58	26.54	0.344	0.210	0.201	100
All 13	22.30	23.60	23.69	27.32	0.303	0.210	0.204	150
U-Net LoRA (6, 7, 8, 16)	21.71	22.88	22.92	26.69	0.292	0.209	0.201	100–250
U-Net full fine-tune (6, 7, 8, 16)	21.71	23.11	–	26.69	0.292	0.182	–	$\leq 4k$
PixArt- α LoRA (all 13)	22.38	23.72	–	27.32	0.285	0.205	–	150
SD 3.5 LoRA (all 13)	22.09	24.00	–	30.61	0.262	0.165	–	250

5 Conclusion

The claims cover 13 unseen maps of one game, one agent, one-tic scoring and mostly one seed. One-tic quality does not guarantee stable rollouts (the supplementary material). Next are other Doom maps and other games, to see whether the same procedure transfers; detecting a shift from the model’s own zero-shot scores so that it repairs itself; closed-loop evaluation of the adapted models; and agents trained inside the adapted model. The pattern we measure, control kept, appearance lost, most of it relearned from a few episodes, is what a deployed world model would need to detect and repair on its own, and DoomShift is a testbed for that loop.

Acknowledgments

[tbd]

References

- [1] D. Valevski, Y. Leviathan, M. Arar, and S. Fruchter. Diffusion models are real-time game engines. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2408.14837>.
- [2] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, and F. Fleuret. Diffusion for world modeling: Visual details matter in Atari. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2405.12399>.
- [3] X. He, C. Peng, Z. Liu, B. Wang, Y. Zhang, Q. Cui, F. Kang, B. Jiang, M. An, Y. Ren, B. Xu, H.-X. Guo, K. Gong, S. Wu, W. Li, X. Song, Y. Liu, Y. Li, and Y. Zhou. Matrix-Game 2.0: An open-source, real-time, and streaming interactive world model. *arXiv preprint arXiv:2508.13009*, 2025.
- [4] R. Po, D. J. Zhang, A. Hertz, G. Wetzstein, N. Wadhwa, and N. Ruiz. MultiGen: Level-design for editable multiplayer worlds in diffusion game engines. *arXiv preprint arXiv:2603.06679*, 2026.
- [5] S. Gao, S. Zhou, Y. Du, J. Zhang, and C. Gan. AdaWorld: Learning adaptable world models with latent actions. In *International Conference on Machine Learning (ICML)*, 2025. URL <https://proceedings.mlr.press/v267/gao25u.html>.
- [6] M. Rigter, T. Gupta, A. Hilmkil, and C. Ma. AVID: Adapting video diffusion models to world models. *Reinforcement Learning Journal*, 6:737–764, 2025. URL <https://rlj.cs.umass.edu/2025/papers/Paper64.html>.
- [7] Y. Chen, J. Yang, Y. Xu, Q. Ma, K. He, P. Li, K. Wang, Z. He, X. Wu, J. Liu, N. Liu, Y. Huang, and L. Wang. XEWorld: Can action-conditioned world models generalize to unseen robot embodiments? *arXiv preprint arXiv:2608.05799*, 2026.
- [8] J. Y. Koh, H. Lee, Y. Yang, J. Baldridge, and P. Anderson. Pathdreamer: A world model for indoor navigation. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021. doi:10.1109/ICCV48922.2021.01447.
- [9] G. Lample and D. S. Chaplot. Playing FPS games with deep reinforcement learning. In *AAAI Conference on Artificial Intelligence*, 2017. doi:10.1609/aaai.v31i1.10827.
- [10] M. Wydmuch, M. Kempka, and W. Jaśkowski. ViZDoom competitions: Playing Doom from pixels. *IEEE Transactions on Games*, 11(3):248–259, 2019. doi:10.1109/TG.2018.2877047.
- [11] Z. Tong, Y. Jin, H. Lai, Z. Wang, Z. Xing, K. Cheng, H. Xu, Z. Pu, S. Zhu, R. Feng, J. Zhao, Y. Zhang, H. Tang, and L. Shao. SCOPE: Simulating cross-game operations in playable environments for FPS world models. *arXiv preprint arXiv:2605.23345*, 2026.
- [12] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, Y. Aytar, S. Bechtle, F. Behbahani, S. Chan, N. Heess, L. Gonzalez, S. Osindero, S. Ozair, S. Reed, J. Zhang, K. Zolna, J. Clune, N. de Freitas, S. Singh, and T. Rocktäschel. Genie: Generative interactive environments. In *International Conference on Machine Learning (ICML)*, 2024. URL <https://proceedings.mlr.press/v235/bruce24a.html>.
- [13] S. Gao, J. Yang, L. Chen, K. Chitta, Y. Qiu, A. Geiger, J. Zhang, and H. Li. Vista: A generalizable driving world model with high fidelity and versatile controllability. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2405.17398>.

- [14] M. Kempka, M. Wydmuch, G. Runc, J. Toczec, and W. Jaśkowski. ViZDoom: A Doom-based AI research platform for visual reinforcement learning. In *IEEE Conference on Computational Intelligence and Games (CIG)*, 2016. doi:10.1109/CIG.2016.7860433.
- [15] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. doi:10.1109/CVPR52688.2022.01042.
- [16] J. Chen, J. Yu, C. Ge, L. Yao, E. Xie, Y. Wu, Z. Wang, J. Kwok, P. Luo, H. Lu, and Z. Li. PixArt- α : Fast training of diffusion transformer for photorealistic text-to-image synthesis. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.00426>.
- [17] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, and R. Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning (ICML)*, 2024. URL <https://proceedings.mlr.press/v235/esser24a.html>.
- [18] T. Salimans and J. Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2202.00512>.
- [19] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://arxiv.org/abs/2010.02502>.
- [20] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. doi:10.1109/CVPR.2018.00068.
- [21] M. Mathieu, C. Couprie, and Y. LeCun. Deep multi-scale video prediction beyond mean square error. In *International Conference on Learning Representations (ICLR)*, 2016. URL <https://arxiv.org/abs/1511.05440>.
- [22] W. Zheng, W. Chen, Y. Huang, B. Zhang, Y. Duan, and J. Lu. OccWorld: Learning a 3D occupancy world model for autonomous driving. In *European Conference on Computer Vision (ECCV)*, pages 55–72, 2024. doi:10.1007/978-3-031-72624-8_4.
- [23] E. Karypidis, I. Kakogeorgiou, S. Gidaris, and N. Komodakis. DINO-Foresight: Looking into the future with DINO. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2412.11673>.
- [24] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- [25] E. Xie, L. Yao, H. Shi, Z. Liu, D. Zhou, Z. Liu, J. Li, and Z. Li. DiffFit: Unlocking transferability of large diffusion models via simple parameter-efficient fine-tuning. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. doi:10.1109/ICCV51070.2023.00390.
- [26] M. E. Taylor and P. Stone. Transfer learning for reinforcement learning domains: A survey. *Journal of Machine Learning Research*, 10(56):1633–1685, 2009. URL <https://www.jmlr.org/papers/v10/taylor09a.html>.